CnA FRAMEWORK

Complete Ecosystem Architecture
Technical Blueprint & Implementation Roadmap

Architecture baseline
PHP 8.4+ · Strict modular design · Provider-independent AI · Vite frontend

Version 1.0 · 13 November 2026

Design principle: a small, stable framework core surrounded by independently installable capabilities.

Document control & executive summary

Item	Decision
Framework	CnA Framework
Runtime	PHP 8.4+ baseline; publish an explicit supported-version matrix.
Architecture	Strict modular, layered, contract-driven architecture.
Databases	MySQL, PostgreSQL and SQLite through PDO adapters and dialects.
Frontend	Vite asset pipeline; TypeScript/vanilla JS baseline; React/Vue optional.
AI	Provider-independent contracts; provider and vector-store adapters optional.
Deployment	Composer install; Node.js needed for build, not normal PHP production runtime.

CnA should be an application platform, not a monolithic framework. Core provides bootstrapping, dependency injection, configuration, HTTP, routing, middleware, validation, events, logging and CLI. Larger capabilities—security tooling, frontend, AI, workflows, administration, deployment and marketplace integrations—are separately installable packages.

This design reduces unnecessary dependencies, supports progressive adoption and gives each subsystem a clear public contract. A package is production-ready only when its API, threat model, tests, upgrade behavior, observability and support policy are documented.

1. Product vision and architecture principles

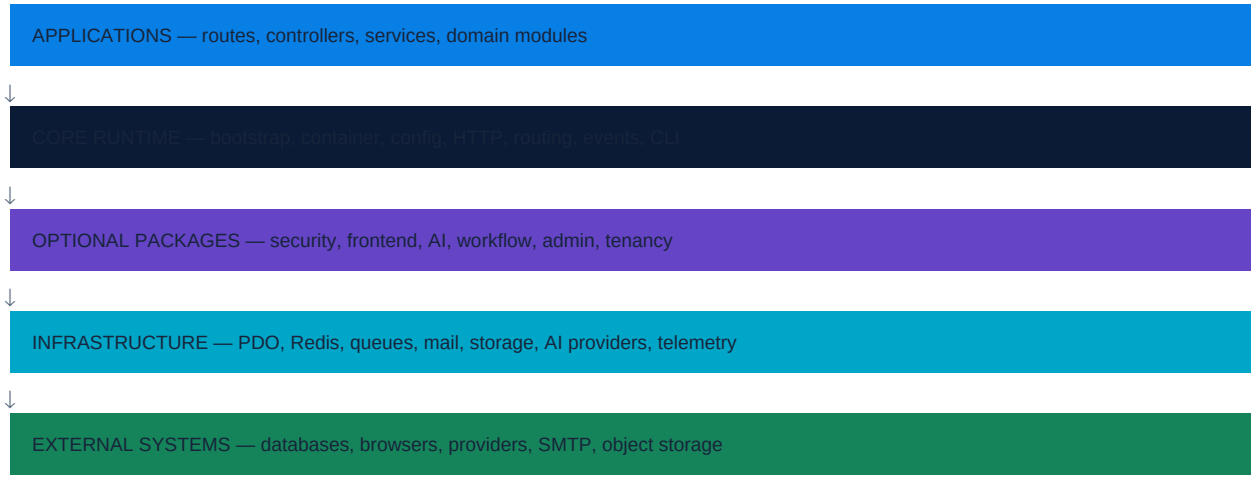
Build a secure, extensible PHP foundation for APIs, SaaS, ERP, CRM and enterprise applications. The framework should make common capabilities consistent without forcing every application to install every subsystem.

- Modularity: independent packages with clear boundaries.
- Dependency inversion: services depend on contracts, not vendor implementations.
- Secure defaults: safe behavior by default; risky overrides are explicit.
- Replaceable infrastructure: database, cache, queue, mail, storage and AI are adapters.
- Observable execution: request, job, query and workflow correlation IDs.
- Backward compatibility: semantic versioning and deprecation policy.
- Progressive adoption: minimal apps do not require AI, Redis, Node, admin or marketplace.

Quality	Target
Security	Threat models, safe defaults, automated security tests.
Performance	Low baseline overhead, profiling and regression benchmarks.
Reliability	Graceful failures, bounded retries, health checks.
Maintainability	Small packages, contracts, static analysis, architecture tests.
Portability	Supported PHP and database matrix tested in CI.

2. System architecture and runtime flow

Four zones: application code, framework core, optional capability packages and infrastructure adapters. Dependencies point toward stable contracts.



- Public entry point loads Composer and creates the CnA application.
- Load environment/configuration, register providers, compile container.
- HTTP kernel creates request context and runs global middleware.
- Router resolves route and route middleware; controller invokes application service.
- Application service applies business rules and calls repository/domain contracts.
- Adapters perform database, cache, queue and external operations.
- Response middleware completes; request-scoped services are released.

Architecture tests must enforce package boundaries and prevent circular dependencies. No package may import another package's internal implementation.

3. Package topology and dependency rules

Family	Packages	Responsibility
Foundation	cna/contracts, container, config	Contracts, DI, configuration.
Runtime	cna/core, http, routing, console	Lifecycle, kernel, routing, middleware, CLI.
Application	cna/validation, events, logging	Validation, events, structured logs.
Persistence	cna/database, query-builder, orm, migrations	PDO, dialects, ORM and schema changes.
Security	cna/auth, authorization, security	Authentication, access control, protections.
Operations	cna/cache, queue, scheduler, monitoring	Caching, jobs, schedules, health, telemetry.
Integration	cna/mail, notifications, storage, webhooks	Communications, files and webhooks.
Frontend	cna/view, frontend, ui	Views, Vite, UI modules and components.
AI	cna/ai-contracts, ai, ai-rag, ai-agents	Providers, orchestration, retrieval and agents.
Business	cna/workflow, multitenancy, feature-flags	Workflow, tenant context, feature control.
Developer/Enterprise	cna/devtools, admin, deploy	Profiler, admin UI and deployment.
Ecosystem	cna/plugin-manager, marketplace-client	Plugin lifecycle and package discovery.

- Contracts package stays dependency-light.
- Core must not depend on Admin, AI providers, Marketplace or UI adapters.
- Optional packages declare contract and adapter dependencies in Composer.
- No network calls during boot unless explicitly configured.
- Extract shared contracts or use events instead of circular package dependencies.

4. Repository and application structure

```
cna-framework/
  composer.json
  packages/{contracts,container,config,core,http,routing,console}
  packages/{database,query-builder,orm,migrations}
  packages/{auth,authorization,security,cache,queue}
  packages/{view,frontend,ai,workflow,multitenancy}
  packages/{admin,devtools,deploy,plugin-manager}
  docs/{architecture,packages,guides,security}
  examples/{minimal-app,api-app,saas-app}
  tests/{unit,integration,contract,security,performance}

my-cna-app/
  app/Providers/
  app/Modules/Users/{Domain,Application,Infrastructure,Http}
  bootstrap/{app.php,providers.php}
  config/  database/{migrations,seeders,factories}
  public/{index.php,build/}
  resources/{views,js,css,modules}
  routes/{web.php,api.php,console.php}
  storage/{cache,logs,sessions,uploads}
  tests/  .env.example  composer.json  package.json  vite.config.js
```

Use a Composer monorepo for coordinated development and independently installable Composer packages for distribution. Keep generated application conventions predictable.

5. Core runtime and application lifecycle

• Application object owns bootstrap state, environment, container and lifecycle.

• Container supports bindings, aliases, safe autowiring, contextual bindings and transient/singleton/scoped lifetimes.

• Configuration repository supports defaults, environment overrides, typed access and secret redaction.

• Service providers have register and boot phases.

• Exception system maps errors safely; production responses hide internals.

• Clock, UUID, filesystem and process abstractions improve testability.

```
Application::create($basePath)
  -> loadEnvironment()
  -> loadConfiguration()
  -> registerProviders()
  -> bootProviders()
  -> handleRequest($request)
  -> terminate($request, $response)
```

Do not run migrations, send messages or perform unexpected network calls during application boot. Request and job scoped services must be cleared after execution.

6. HTTP, routing and API architecture

• HTTP kernel with deterministic global, route and controller middleware.

• Route groups, named routes, URL generation, constraints and parameter binding.

• Controller dispatch through DI; consistent response normalization.

• Exception-to-response mapping and structured access logs.

• API versioning, validation, pagination, filtering, sorting and resource serialization.

• OpenAPI generation from explicit route/DTO metadata.

• Idempotency keys for retryable operations; webhook signature and replay protection.

```
$router->group(['prefix' => '/api/v1', 'middleware' => ['api']], function ($r) {
    $r->get('/users', [UserController::class, 'index'])
      ->name('api.v1.users.index');
});
```

Route metadata should be inspectable by CLI, OpenAPI generators and security audit tools.

7. Database, ORM and query intelligence



- PDO drivers and explicit capability detection for all supported engines.
- Parameterized queries by default; validate/quote identifiers through dialects.
- Transactions, savepoints where supported and safe deadlock retry.
- Migrations with locking, status, dry-run and production safeguards.
- ORM relations, eager loading, pagination, casts and soft deletes.
- Read/write routing and TLS/timeouts where supported.
- Query intelligence: normalized SQL fingerprints, duration, repeated relation queries, query budgets and lazy-loading prevention.

Never auto-rewrite arbitrary SQL in production. Show evidence and safe recommendations. Backup/restore and schema diff belong in optional tooling and require explicit target confirmation.

8. Security Engine

Area	Capabilities
Authentication	Sessions, API tokens, password hashing, MFA hooks, recovery.
Authorization	RBAC, policies, resource checks, optional ABAC.
Request protection	CSRF, secure cookies, CORS controls, rate and body limits.
Output protection	Escaping, CSP, headers and safe serialization.
Data protection	Encryption interfaces, key rotation hooks, secret redaction.
File handling	Content checks, limits, randomized names, safe storage, traversal defense.
Audit	Actor/context metadata, security events and export.
Audit CLI	Configuration, dependency, headers, permissions and deployment checks.

- Threat-model auth bypass, broken object authorization, CSRF/XSS/SQLi/SSRF/path traversal.
- Test session fixation, token leakage, weak recovery and cookie settings.
- Test tenant boundaries across database, cache, queue, files and AI retrieval.
- Protect against webhook replay, plugin risk and AI tool abuse.
- Use OWASP ASVS as a verification reference; do not claim certification without assessment.

```
php cna security:audit
php cna security:dependencies
php cna security:headers
php cna security:permissions
php cna security:report --format=json
```

9. Frontend Engine and Vite

Component	Responsibility
View Engine	Layouts, templates, components, escaping.
Asset Manager	Resolve logical entries and render CSS/JS tags.
Manifest Reader	Resolve imports, CSS, preload and deduplication.
Dev Server Adapter	HMR client and trusted development origin.
UI Modules	Feature-specific TS/JS/CSS and mount points.
UI Kit	Optional design tokens, accessible components and forms.
React/Vue adapters	Optional integrations; never required by core.

- Development: PHP serves routes; Vite serves assets/HMR on trusted origin.
- Production: hashed assets and manifest; PHP resolves tags.
- Production PHP must not require Node.js or a running Vite server.
- Restrict manifest paths to build root; escape generated attributes.
- SSR and SPA routing are later phases after asset integration is stable.

```
resources/
  views/{layouts,components,pages}/
  js/app.ts  css/app.css
  modules/{dashboard,users}/
  vite.config.js
public/build/

Vite::assets('app');
Vite::assets('dashboard');
```

10. AI Engine

Subsystem	Scope
Provider registry	Provider adapters, model capabilities, credentials.
Chat/streaming	Typed messages, stream events, timeout, cancellation.
Structured output	Schema validation and typed result mapping.
Embeddings/RAG	Ingestion, chunking, embeddings, retrieval and citations.
Agent runtime	Allowlisted tools, policy checks, limits, approval.
Prompt management	Versioning, test cases, rollout controls.
Observability	Token usage, cost estimates, latency and redacted traces.

- Tools must be allowlisted and have schemas, permissions, timeouts and audit behavior.
- Require approval for sensitive actions; enforce per-user/tenant budgets.
- Treat retrieved documents and model outputs as untrusted input.
- Never grant unrestricted shell, filesystem or database access.
- RAG retrieval must enforce the source data's authorization rules.
- Vector stores and providers are optional adapters.

```
Documents -> Access filtering -> Chunking -> Embeddings
-> Vector store + metadata -> Permission-filtered retrieval
-> Context with source references -> Model -> Response + citations
-> Evaluation and telemetry
```

11. Workflow Engine and background processing

• Versioned workflow definitions as state machines or declarative graphs.

• Sequential, conditional, parallel and approval steps.

• Persist execution state; pause, resume, cancel and retry.

• Durable step checkpoints and idempotency keys.

• Queue contract: reserve, acknowledge, release, delay and dead-letter.

• Workers: graceful shutdown, bounded concurrency, timeout and health.

• Scheduler: overlap prevention, distributed locks and run history.

Assume at-least-once job delivery and make handlers idempotent. Do not claim exactly-once processing across distributed systems. Use an outbox pattern when database writes and message publication must remain consistent.

12. Plugin system and marketplace

```
Discover metadata -> Validate compatibility/permissions
-> Resolve Composer dependencies -> Register provider
-> Explicit install/migrations -> Enable -> Monitor/update
-> Disable/uninstall with data-retention safeguards
```

- Use Composer/Packagist as initial distribution foundation.
- Marketplace initially indexes metadata, compatibility and security status.
- Use immutable version references and dependency audit.
- Display maintainer, source, license, supported PHP/CnA versions.
- PHP plugins execute with host application privileges; do not claim sandboxing.
- Require explicit install/enable/disable/uninstall and migration checks.

13. Admin Studio and observability

Module	Functions
Overview	Health, version, environment, key indicators.
Users/access	Users, roles, permissions, sessions and history.
Jobs/scheduler	Queue depth, worker status, failed jobs, schedule runs.
Logs/exceptions	Searchable structured logs with redaction.
Database	Connection and migration status, query summaries.
Security	Audit findings, dependency alerts and events.
AI/workflows	Usage, provider health, executions, approvals and failures.

- Metrics: request rate, latency, errors, queue depth, job duration and DB timing.
- Logs: structured JSON with request/trace IDs and redaction.
- Traces: OpenTelemetry-compatible spans and context propagation.
- Separate liveness and readiness checks.
- Admin routes disabled until explicitly enabled; strong auth, authorization, rate limits and audit.
- Never expose debug tools, environment secrets or raw stack traces publicly.

14. Enterprise and multi-tenancy

Strategy	Isolation	Trade-off
Shared DB + tenant_id	Logical row isolation	Lower cost; every query and unique constraint must be tenant-aware.
Schema per tenant	Schema-level isolation	More separation; migration and connection complexity.
Database per tenant	Separate database	Operational separation; higher provisioning/maintenance.

- Resolve tenant from verified domain, token claim or trusted mapping.
- Validate tenant status before creating context.
- Bind tenant context to request/job scope and clear after execution.
- Apply tenant filtering to repositories, cache, files, queue, AI and audit.
- Do not trust tenant IDs supplied as arbitrary request parameters.
- Test cross-tenant isolation in all asynchronous and data access paths.

Subscription/billing adapters, usage metering, quotas, custom domains, branding, SSO and export should be optional enterprise packages.

15. Deployment and operations

• Build frontend assets in CI; deploy compiled output.

• Install Composer dependencies from a lockfile with production settings.

• Run migrations as an explicit release step with backup and rollback plan.

• Keep secrets outside source control.

• Run queue workers/schedulers as managed processes with health checks.

• Use immutable releases or atomic symlink swaps where possible.

• Keep persistent uploads outside ephemeral release directories.

Deploy Manager	Requirement
Preflight	PHP extensions, permissions, env, DB, frontend manifest.
Release	Release ID, commit, lockfile and artifact checksum.
Migrations	Pending change preview and explicit production confirmation.
Health	Readiness, workers and critical dependency checks.
Rollback	Previous app release; document DB rollback limitations.
Audit	Actor, timestamp, change and deployment result.

16. CLI and developer experience

Family	Commands
App	cna app:info, about, env:check
Generators	make:controller, make:model, make:module, make:plugin
Database	migrate, migrate:status, rollback, db:backup, db:restore
Frontend	frontend:install, frontend:dev, frontend:build, frontend:status
Security	security:audit, security:dependencies, security:report
Operations	queue:work, queue:failed, schedule:run, health:check
AI	ai:providers, ai:usage, ai:prompt:test
Workflow	workflow:list, workflow:run, workflow:inspect
Diagnostics	route:list, config:show, profile:request

- Support --help, --verbose, --no-interaction and --format=json where useful.
- Destructive commands require confirmation or deliberate --force.
- Return meaningful exit codes and never print secrets.
- Provide dry-run for migrations, deployments and plugin changes.

17. Configuration, events and integrations

• Ship documented config with safe defaults and environment overrides.

• Validate configuration at boot/preflight with actionable errors.

• Support external secret providers through adapters; never log secret values.

• Synchronous events for local decoupling; queued events for async work.

• Version public event payloads; document retry/idempotency behavior.

• Contracts cover cache, queue, mail, storage, notifications, webhooks and telemetry.

Integration adapters must define timeout, error, retry, resource ownership and capability semantics.

18. Testing, quality and CI/CD

Layer	Verifies
Unit	Pure classes, policies and deterministic logic.
Contract	Adapters conform to interface and error semantics.
Integration	Database, queue, cache, storage and providers.
Architecture	Boundaries, dependency direction and no cycles.
Security	Authorization, CSRF, injection defense, tenancy and redaction.
Performance	Bootstrap, routing, ORM, serialization and memory.
Compatibility	Supported PHP versions and DB engines.
End-to-end	Generated app, frontend build and deployment smoke tests.

- Use PHPUnit or Pest; PHPStan/Psalm; PHP-CS-Fixer/PHP_CodeSniffer.
- Composer audit and reviewed dependency updates.
- Benchmark critical paths and track regressions.
- CI: PHP matrix, static analysis, standards, tests, DB matrix, audit, frontend build, performance smoke, artifact and release notes.
- Do not merge breaking contracts or weakened security checks without approved exception.

19. Threat model and operational safeguards

Threat	Safeguard
Untrusted HTTP input	Validation, encoding, parameterized SQL, size limits.
Broken authorization	Deny-by-default policies, resource checks and negative tests.
Tenant leakage	Scoped context, tenant-aware adapters and isolation tests.
Supply-chain compromise	Lockfiles, audits, provenance, review and minimal deps.
Malicious plugin	Trusted source, explicit install; no sandbox claims.
AI tool abuse	Allowlist, schemas, approvals, budgets and audit.
Secret exposure	Redaction, external secrets and log tests.
Unsafe deployment	Preflight, immutable artifacts, migration confirmation.
Denial of service	Rate limits, bounded workers, timeouts and backpressure.

- Production debug mode off; secure cookie defaults documented.
- CSRF enabled for cookie-authenticated state changes.
- CORS restrictive by default; no wildcard with credentials.
- Trusted proxies explicitly configured.
- File uploads stored outside executable public paths by default.
- Admin/diagnostic endpoints protected or disabled.
- Key rotation and incident response documented.

20. Roadmap and release gates

Phase	Deliverables	Exit criteria
0 Foundation	Contracts, DI, config, lifecycle, logging, CLI	Minimal app boots; CI and architecture tests.
1 Web core	HTTP, router, middleware, validation, errors	CRUD/API app with request tests.
2 Persistence	PDO, dialects, query builder, ORM, migrations	DB matrix and transaction/N+1 tests.
3 Security	Auth, policies, CSRF, sessions, rate limits, audit	Threat model and security tests.
4 Dev tools	Generators, profiler, route inspector, OpenAPI	Generated code passes checks.
5 Frontend	Views, Vite, UI modules	HMR + production manifest; no Node runtime.
6 Operations	Cache, queue, scheduler, monitoring	Worker retries and telemetry verified.
7 Enterprise	Tenancy, workflow, flags, SSO adapters	Isolation and durable workflow tests.
8 AI	Providers, streaming, RAG, agents	Permission, cost and abuse tests.
9 Ecosystem	Plugin manager, Admin, marketplace, deploy	Trust, security and compatibility policy.

P0: stable core. P1: production essentials. P2: developer advantage. P3: platform expansion. Do not start with Admin Studio, AI agents or Marketplace before the runtime contracts stabilize.

- Every package has public API docs, dependency review and tests.
- Security/failure modes and configuration documented.
- Upgrade/migration behavior and CLI help covered.
- Static analysis and compatibility matrix pass.
- Changelog and semantic-version impact reviewed.

21. Decisions to finalize before implementation

Decision	Recommended starting point	Question
PHP support	8.4+ baseline	Which newer PHP versions enter each release matrix?
HTTP contracts	PSR-compatible adapters	Native abstraction or PSR-7/15 integration?
ORM	Repository-friendly ORM + query builder	Active Record, Data Mapper or hybrid?
Queue	Database driver first; Redis optional	Which delivery/retry guarantees?
Frontend	Vite + TypeScript + vanilla JS	Which adapters receive official support?
Admin	Separate optional package	Embedded package or separately deployed app?
Tenancy	Tenant context contract first	Which isolation strategies are supported?
AI	Provider capability contracts	How are model differences exposed?
Plugins	Composer packages + explicit review	How are trust and provenance assessed?
Release	SemVer + compatibility matrix	Support window and LTS policy?

Record each choice in an Architecture Decision Record: context, options, decision, consequences, security impact and migration plan.

First milestone: Composer-installable minimal app with container, config, providers, HTTP kernel, router, middleware, structured errors, CLI and tests. Add advanced capabilities only after this foundation is dependable.

Appendix A. Example package and API conventions

```
{
  "name": "cna/http",
  "type": "library",
  "require": {"php": "^8.4", "cna/contracts": "^1.0"},
  "autoload": {"psr-4": {"CnA\\\\Http\\\\": "src/"}}
}
```

- Use strict types and typed return values.
- Prefer immutable value objects for request metadata and event payloads.
- Use explicit exceptions with stable error codes.
- Avoid boolean-parameter APIs that hide behavior.
- Use capability interfaces for optional driver features.
- Document resource ownership, cleanup, timeout and retry semantics.

```
interface ServiceProviderInterface {
  public function register(ContainerInterface $container): void;
  public function boot(ApplicationInterface $app): void;
}
```

Appendix B. Reference documentation

Reference	Purpose
https://www.php.net/docs.php	PHP language and runtime.
https://getcomposer.org/doc/	Composer packages and autoloading.
https://www.php-fig.org/psr/	PHP interoperability standards.
https://vite.dev/guide/	Vite development and production builds.
https://owasp.org/www-project-application-security-verification-standard/	Security verification planning.
https://cheatsheetseries.owasp.org/	Web application security guidance.
https://opentelemetry.io/docs/	Metrics, traces and context propagation.
https://spec.openapis.org/oas/latest.html	API descriptions.
https://phpunit.de/documentation.html	Testing documentation.

Final recommendation: build CnA as a stable modular core with optional first-party packages. Prioritize runtime, HTTP, persistence, security and developer experience before workflows, AI, enterprise management and marketplace features. This sequencing reduces risk and gives each future subsystem dependable contracts.